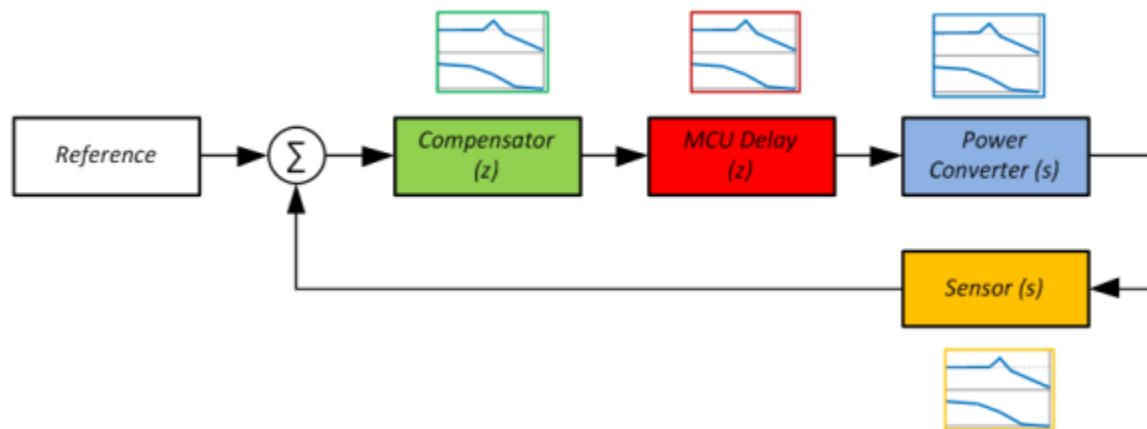


CONTROLLER DESIGN FOR POWER CONVERSION SYSTEMS

Introduction



A typical feedback system found in power converters

Switched-mode power converters generally use **PI**, **2p2z**, or **3p3z** feedback compensators to regulate currents and voltages. Motor drives also use these to regulate mechanical states such as torque (which is mostly proportional to current in the windings), angular velocity, and occasionally angular position. There are many distinct applications; each is different with respect to regulated variables. These examples are but a mere sample from the myriad of possibilities found in modern power electronics:

1. DC/DC Buck power supply – internal inductor current, output voltage
2. Grid-connected inverter – output (grid) current, internal DC link voltage
3. Fuel-cell DC/DC converter – input (fuel cell) current
4. Ultracapacitor test system – capacitor current, capacitor voltage
5. Motor drives – phase winding current, DC link stabilization

The three aforementioned compensators – **PI**, **2P2Z** and **3P3Z** have different properties – different number of poles and zeroes. Poles are generally used to attenuate controlled signal at higher frequencies, while zeroes are used to boost signal amplitude. Similarly, poles tend to delay (lag) the signal whereas zeroes tend to advance the signal (lead).

Power converters virtually always have to be compensated to shift their poles (eigenvalues) to desired locations, both to enhance dynamics and to separate current and voltage dynamics. **High-**

performance feedback control is also needed to maintain regulated voltages and currents within converter operating window and to compensate for any open-loop transfer function error and external disturbances.

The typical conditions of stability for single-input single-output power conversion system are twofold:

1. The phase margin is more than 0 degrees (this is somewhat connected to the requirement of -20 dB roll-off in the open-loop transfer function at the crossover frequency).
2. Right half-plane zeroes (if any) are above the crossover frequency as those are difficult to compensate.

A good compensator provides significant gain at low frequencies (to have good command following up to the crossover frequency) and significant attenuation at high frequencies to increase noise immunity.

Physical reasoning behind positive phase margin:

Imagine a position control system whose reference setpoint follows a sinusoidal trajectory. Signal error is calculated as reference minus measured signal. If the measured signal is delayed more than 180 degrees (= has negative phase margin), then the subtraction from reference will make the drive signal in phase with the reference signal and thus changing the negative (stabilizing) feedback into positive (destabilizing) feedback.

Compensator Design in Laplace domain

For example, let's say we need to control phase current in a brushless permanent-magnet motor. It is desired to control the average current and not the peak current. How do we design a proper compensator? According to the following (standard) methodology for average current control:

1. Select the desired crossover frequency. The maximum crossover frequency should be about $f_{sw}/5$ but more typically the crossover frequency of the fastest loop is placed at $f_{sw}/10$.
2. Place a pole at half the switching frequency to attenuate current switching harmonics.
3. Place a zero at or around the desired crossover frequency to obtain a reasonable phase margin.
4. Place an additional pole to the origin to obtain zero steady-state error.
5. Adjust proportional gain K_c to set open-loop gain to 0 dB at the specified crossover frequency.

The continuous-time compensator is then of the following form:

$$H(s) = \frac{K_c s + \omega_z}{s s + \omega_p}$$

We can use the Tustin's method to convert this continuous compensator to its digital form via the substitution below (Why Tustin? Scroll to the bottom of this post :)):

$$s = \frac{2}{T_s} \frac{z-1}{z+1}$$

And the resulting compensator in the digital domain is:

$$H(z) = K_p T_s \frac{T_s \omega_z z^2 + 2T_s \omega_z z + T_s \omega_z + 2z^2 - 2}{2(T_s \omega_p z^2 - T_s \omega_p + 2z^2 - 4z + 2)}$$

Algebraic simplification was done with Python:

```

1
2     from sympy import *
3     from sympy import pprint, collect
4     from sympy.abc import z
5
6     Kp = Symbol('Kp');
7     wp = Symbol('wp');
8     wz = Symbol('wz');
9     Ts = Symbol('Ts');
10
11     s = 2/Ts*(z-1)/(z+1)
12
13     H = Kp/s*(1+s*wz)/(1+s*wp)
14     pprint(simplify(collect(expand(H), z)))

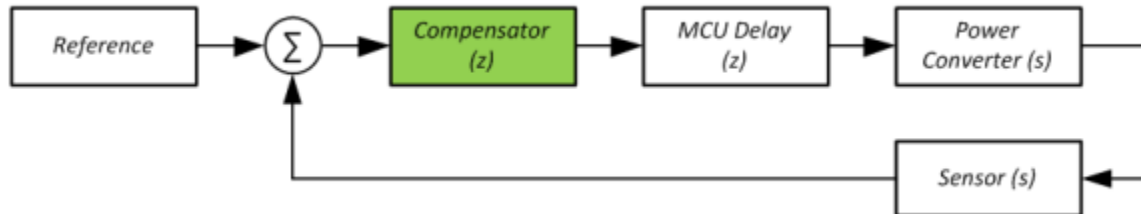
```

As we can see, a PI compensator with an additional pole has the same discrete form as the aforementioned 2p2z compensator:

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}} = \frac{b_0 z^2 + b_1 z^1 + b_2}{z^2 + a_1 z^1 + a_2}$$

Note: the 2p2z compensator is also called a digital biquad filter in the digital signal processing jargon.

Where is the compensator placed and how is it implemented?



Integrator saturation is a critically important technique that disables error accumulation in case the compensator output is saturated. The saturation is typically caused by a physical constraint – such as maximum supported current by power devices or the nonexistence of negative synthesizable voltage. Alike, realistic duty ratio for a power switch is between 0 % and 100 %. Can integrator saturation be implemented in the 2p2z compensator as easily as it can be implemented in the PI compensator alone? The 2p2z compensator can be implemented in the form below (so called **Direct Form II**). The integrator term is unfortunately quite obfuscated. Saturation of one memory block affects the whole transfer function. It does not seem possible to easily saturate only the PI compensator with this digital structure.

In light of these findings, let's expand the original compensator transfer function:

$$H(s) = \frac{K_c (s + \omega_z)}{s (s + \omega_p)} = \frac{K_c (s + \omega_z)}{s \omega_p} \frac{\omega_p}{(s + \omega_p)}$$

The right side of the equation above separates the high-frequency poles from the PI compensator. The PI compensator is not in a form suitable for implementation – due to the differentiation term. Let's expand the PI compensator transfer function:

$$H(s) = \frac{K_c}{s \omega_p} (s + \omega_z) = \underbrace{\frac{K_c \omega_z}{\omega_p}}_{\text{Proportional term}} + \underbrace{\frac{K_c}{\omega_p}}_{\text{Integral term}} \frac{1}{s}$$

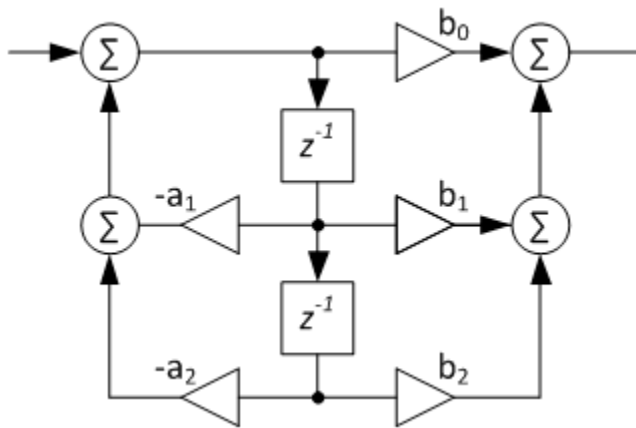
$$\frac{K_c}{\omega_p}$$

The term $\frac{K_c}{\omega_p}$ is equal to the integral gain; the product of this integral gain and the location of the numerator “zero” is equal to the proportional term. The frequency of the “zero” is equal to the location of the “knee” characteristic for PI compensators.

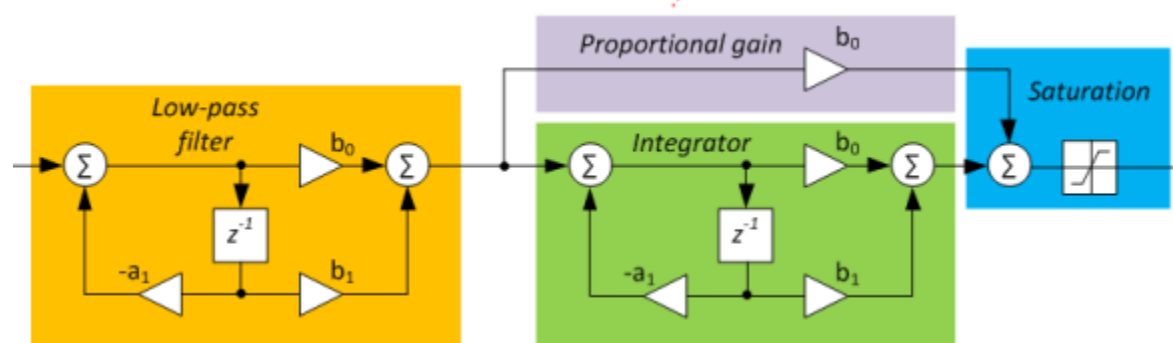
Note that the signal filter is not in the feedback but in the forward path. The benefit of this placement is the attenuation of high frequency harmonics in both the measured and commanded paths.

s-domain	z-domain
K_c	K_c
$\frac{1}{s}$	$\frac{T_s}{2} \frac{z+1}{z-1}$

Direct Form II implementation of the 2p2z (PI + LPF) discretized transfer function follows:



Alternatively, the compensator can be implemented in the **separate form** as shown below. Notice the **saturation block** at the output- an anti-windup would be fed to the integrator block input. Both forms are quite easy to implement in the C language.



Design Example

Let's design a current controller for a boost converter operating in the continuous current mode (inductor current does not **stay** at zero). Using the small signal perturbation analysis we obtain the transfer function from duty ratio to current:

$$H(s) = \frac{\hat{i}}{d} = \frac{V_{out}}{Ls+R}$$

The power converter has these parameters:

Switching frequency [Hz]	100e3
Inductance [H]	100e-6
Inductor resistance [Ohm]	0.01
Input voltage [V]	100
Output voltage [V]	240

Following the aforementioned design methodology we obtain the following continuous-time compensator:

```

1  H_compensator =
2
3  9600 s + 9.6e07
4  -----
5  s^2 + 3.142e05 s

```

This can be separated into two parts:

```

1  H_pi =
2
3  0.03056 s + 305.6
4  -----
5  s

```

and

```

1  H_filter =
2
3  3.142e05
4  -----
5  s + 3.142e05

```

with respective discrete-time counterparts:

```

1  H_compensator_digital =
2
3      0.0196 z^2 + 0.001867 z - 0.01774
4      -----
5      z^2 - 0.778 z - 0.222

```

The PI compensator transfer function:

```

1  H_pi_digital=
2
3      0.611 z + 0.611
4      -----
5      z + 0.222

```

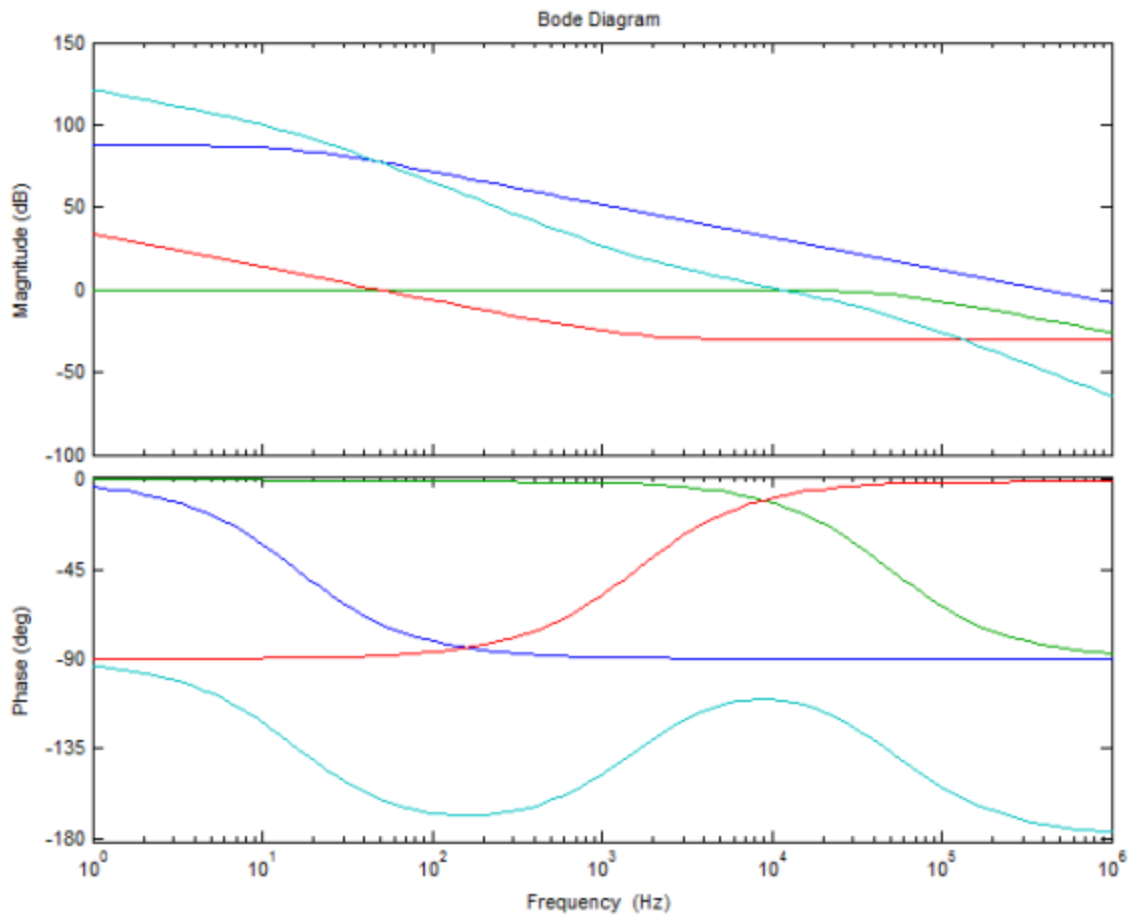
and the low-pass filter:

```

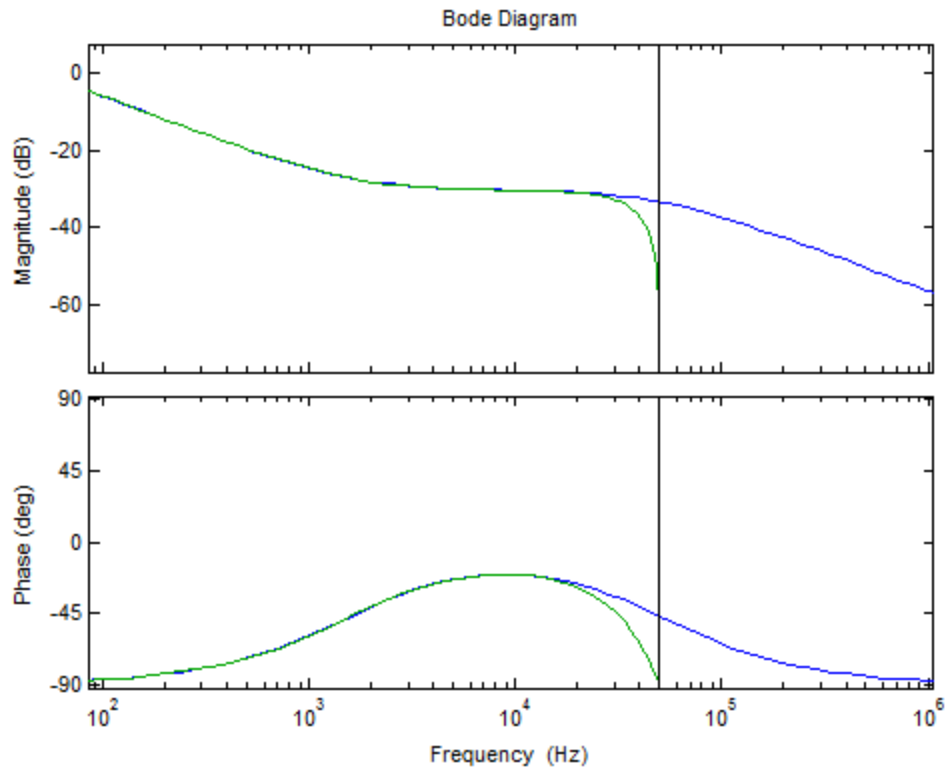
1  H_filter_digital=
2
3      0.03209 z - 0.02903
4      -----
5      z - 1

```

The Bode plots below shows BLUE – plant, RED – PI, green – LPF, teal – complete open-loop transfer function (all continuous). Note that the crossover frequency (TEAL) is 10 kHz.



And finally the second Bode plot shows the difference between the continuous (BLUE) and discrete (GREEN) compensators. Note that frequencies of our interest are just below ~ 20 kHz or so since the crossover frequency is 10 kHz. The gain and phase characteristics between $H(s)$ and $H(z)$ tend to diverge above 20 kHz.



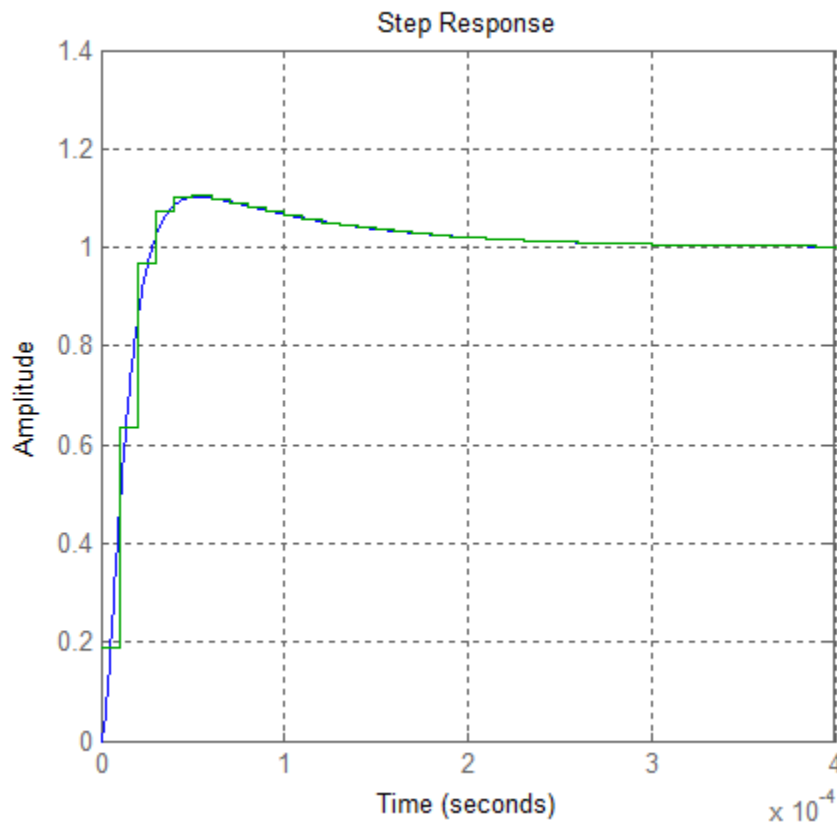
Final compensator gains:

		s-domain	z-domain
Proportional gain	$K_p = K_c / w_p$	0.0305577	0.0305577
Integrator gain	$K_i = K_c / w_p * w_z$	305.6	X
A0	1	X	1
A1	-1	X	-1
B0	$K_i T_s / 2$	X	0.001528
B1	$K_i T_s / 2$	X	0.001528
LPF			
A0	1	X	1
A1	$(w_p T_s - 2) / (w_p T_s + 2)$	X	0.2220309407
B0	$w_p T_s / (2 + w_p T_s)$	X	0.61101547035

B1	$w_p T_s / (2 + w_p T_s)$	X	0.61101547035
----	---------------------------	---	---------------

The resulting phase margin is about 69 degrees. Closed-loop unit-step response is shown below (command increases from 0 A to 1 A):

Note: The design should be further optimized. 10% overshoot is quite significant, I have seen it set to 5% (to get higher rise time) or none (say regulator output voltage cannot overshoot).



References

[Kassakian – Principles of Power Electronics](#)

And of course [Jason's blog](#) as he has been a great inspiration to me.

Interesting Notes A – Why Tustin?

Which discretization method should we use? There are a few- ZOH, FOH, Tustin (bilinear), impulse-invariant, and zero-pole matched. Let's take a look at the two which are most common: ZOH and bilinear.

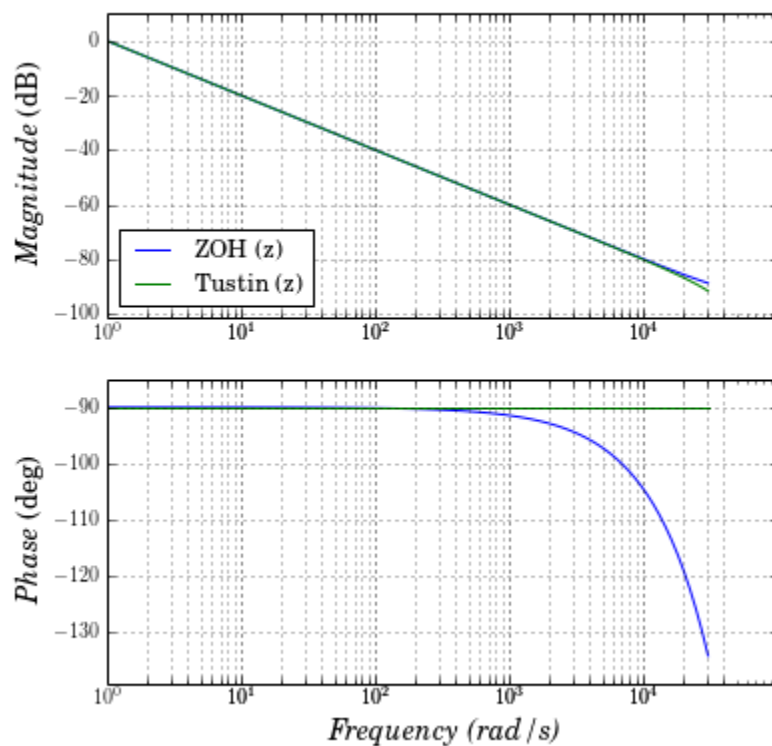
s-domain	z-domain – bilinear	z-domain – zoh
----------	---------------------	----------------

$$\frac{1}{s}$$

$$\frac{T_s}{2} \frac{z+1}{z-1}$$

$$\frac{T_s}{z-1}$$

Why do we have different discrete time representations of the same continuous-domain function? Well, all discretization methods are a form of approximation. The continuous time domain system representation contains information for all frequencies (from minus infinity to plus infinity). Discrete time systems, on the other hand, start to deviate as the system frequency approaches the Nyquist frequency.



Sampling frequency in this particular case is 20 kHz, which sets the maximum signal frequency at 10 kHz (or 62.8 krad/s). The magnitude plots favors the “ZOH” approximation close to the Nyquist frequency but the phase plot shows a large deviation from the original continuous-time domain model at frequencies well below the Nyquist frequency. Accurate phase lag is very important to us since lower phase margin results in overshoot, oscillations, and possibly even instability. Hence, we will choose the Tustin approximation from now onwards.

Interesting Notes B- Low-pass filter discretization

Laplace domain transfer function of a first-order low-pass filter is:

$$H_{LPF}(s) = \frac{\omega_p}{s + \omega_p}$$

Its discrete time implementation is:

$$H_{LPF}(z) = \frac{T_s \omega_p (z+1)}{T_s \omega_p (z+1) + 2z - 2} = \frac{b_0 z + b_1}{z + a_1}$$

Source : <http://tomontheroll.wordpress.com/2014/07/22/controller-discretization-for-power-conversion-systems/>